

```
Counts {cases = 1, tried = 1, errors = 0, failures = 0}
```

Consider a situation when there is a failure:

```
import Test.HUnit

test1 = TestCase $ assertBool "Failure!" (False && False)
```

The corresponding output is shown below:

```
ghci> runTestTT test1

### Failure:
Failure!
Cases: 1 Tried: 1 Errors: 0 Failures: 1
Counts {cases = 1, tried = 1, errors = 0, failures = 1}
```

assertFailure and *assertString* functions take a string as input, and return an assertion. These are used as part of other test functions. The *assertFailure* function is used in the definition of the *assertBool* function as shown below:

```
-- | Asserts that the specified condition holds.
assertBool :: String -- ^ The message that is displayed
              if the assertion fails
              -> Bool -- ^ The condition
              -> Assertion
assertBool msg b = unless b (assertFailure msg)
```

You can also use *assertString* for handling a specific case. For example:

```
import Test.HUnit

test1 = TestCase $ assertString "Failure!"
```

Executing the above code in GHCi results in the following failure message:

```
ghci> runTestTT test1

### Failure:
Failure!
Cases: 1 Tried: 1 Errors: 0 Failures: 1
Counts {cases = 1, tried = 1, errors = 0, failures = 1}
```

Hspec is another testing framework for Haskell, similar to Ruby's RSpec. You can install it on any GNU/Linux distribution using the Cabal tool:

```
$ cabal update && cabal install hspec hspec-contrib
```

A simple example is shown below:

```
import Test.Hspec
```

```
main :: IO ()
main = hspec $ do
  describe "Testing equality" $ do
    it "returns 3 for the sum of 2 and 1" $ do
      3 `shouldBe` (2 + 1)
```

Executing the above code with GHCi produces the following verbose output:

```
$ ghci spec.hs

GHCi, version 7.6.3: http://www.haskell.org/ghc/ :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main             ( spec.hs, interpreted )
Ok, modules loaded: Main.

ghci> main
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package HUnit-1.2.5.2 ... linking ... done.
Loading package old-locale-1.0.0.5 ... linking ... done.
Loading package time-1.4.0.1 ... linking ... done.
Loading package random-1.0.1.1 ... linking ... done.
Loading package transformers-0.3.0.0 ... linking ... done.
Loading package stm-2.4.2 ... linking ... done.
Loading package bytestring-0.10.0.2 ... linking ... done.
Loading package unix-2.6.0.1 ... linking ... done.
Loading package containers-0.5.0.0 ... linking ... done.
Loading package pretty-1.1.1.0 ... linking ... done.
Loading package template-haskell ... linking ... done.
Loading package QuickCheck-2.6 ... linking ... done.
Loading package ansi-terminal-0.6.2.1 ... linking ... done.
Loading package async-2.0.1.6 ... linking ... done.
Loading package hspec-expectations-0.6.1 ... linking ... done.
Loading package quickcheck-io-0.1.1 ... linking ... done.
Loading package setenv-0.1.1.1 ... linking ... done.
Loading package primitive-0.5.0.1 ... linking ... done.
Loading package tf-random-0.5 ... linking ... done.
Loading package hspec-core-2.0.2 ... linking ... done.
Loading package hspec-discover-2.0.2 ... linking ... done.
Loading package hspec-2.0.2 ... linking ... done.

Testing equality
  returns 3 for the sum of 2 and 1

Finished in 0.0002 seconds
1 example, 0 failures
```

The keywords *describe* and *it* are used to specify the tests. The *context* keyword can also be used as an alias for *describe*.